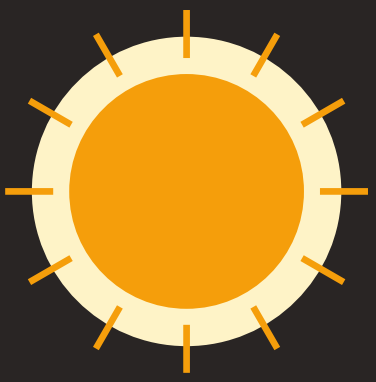

SOLAR BREAK

AI-Powered Solar Advisory Agent — Technical Specification

Powered by LangChain · Ollama · RAG · Web UI

STATES COVERED

Leh · Maharashtra · Gujarat · Rajasthan · Karnataka · Telangana

Version 1.0 · October 2026 · University Project

CONFIDENTIAL — FOR ACADEMIC USE ONLY

TABLE OF CONTENTS

01 Agent System Flowchart
02 Data Procurement Guide
03 Data Storage Architecture
04 Web UI Features & Design
05 Tech Stack Summary

01

Agent System Flowchart

The Solar Break agent is a multi-chain conversational system. User inputs flow through a session memory layer into an intent router, which dispatches queries to one of four specialised chains. All chain outputs are merged by a response synthesiser before delivery. A follow-up loop allows multi-turn conversation within the same session.

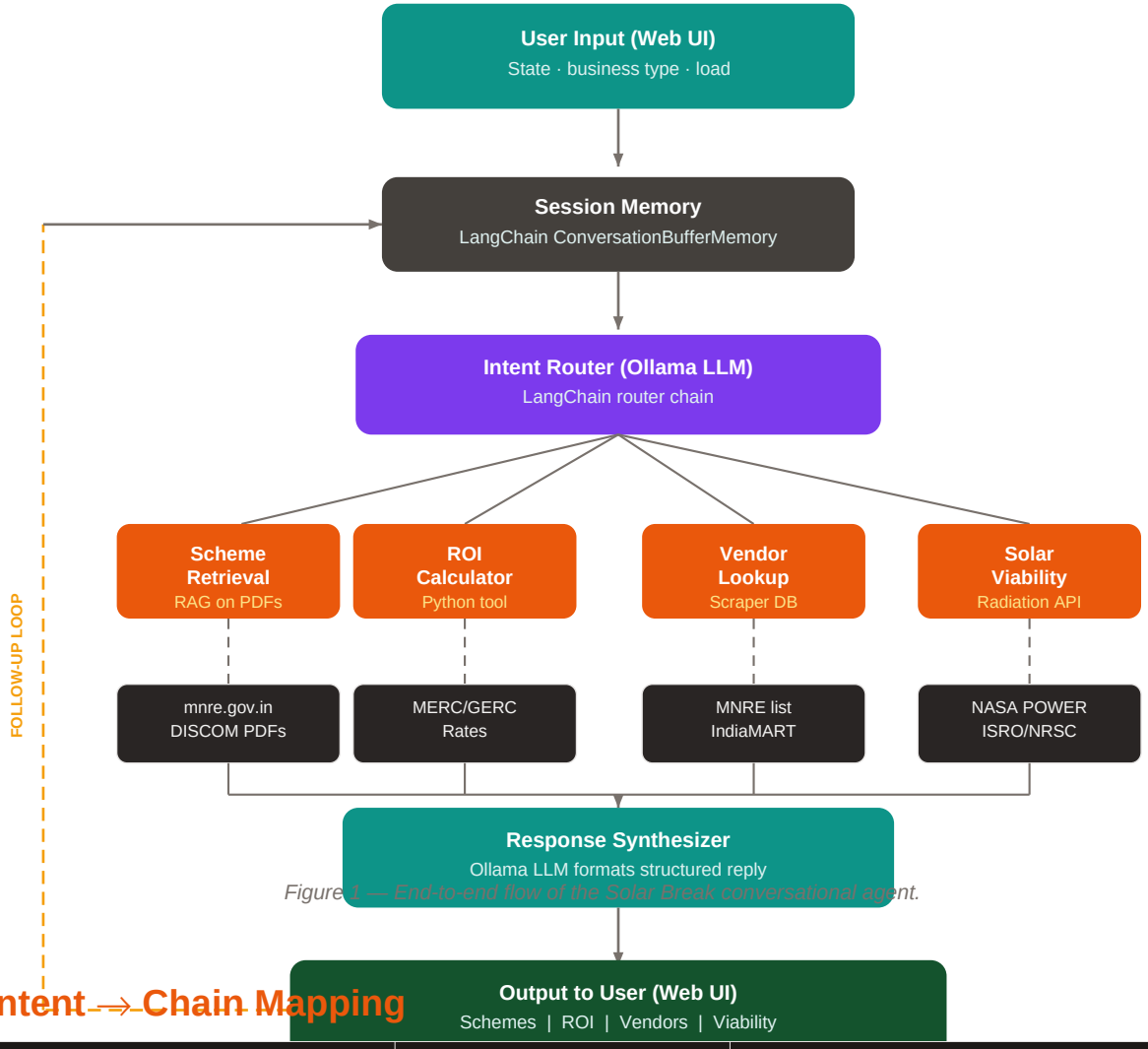

Figure 1 — End-to-end flow of the Solar Break conversational agent.

Intent → Chain Mapping

What subsidies do I get?	Scheme Retrieval	State + Central scheme list
What is my ROI?	ROI Calculator	Payback period, 10-yr savings
Where do I buy panels?	Vendor Lookup	Rated vendor list by state
Is my location suitable?	Solar Viability	GHI score, peak months

02

Data Procurement Guide

The agent requires six distinct data categories. Each has a specific source, acquisition method, update cadence, and storage target. Below is a complete breakdown per category.

2.1 Government Schemes (Central + State)

Field	Detail
Sources	mnre.gov.in, solarrooftop.gov.in, MSEDCL (MH), GUVNL (GJ), RRECL (RJ), KREDL (KA), TSREDCO (TS), LR
Method	Manual PDF download + automated scraper (BeautifulSoup/Scrapy) for news/updates
Key fields	Scheme name, eligible entity type, subsidy %, max capacity (kW), validity date, application link
Update cadence	Monthly re-scrape; government budgets typically update April & October
Volume	~200-400 scheme records across 6 states

2.2 Solar Radiation & Irradiance Data

Field	Detail
Primary source	NASA POWER API (power.larc.nasa.gov/api/temporal/daily/) — free REST, no key needed
Secondary source	ISRO/NRSC Bhuvan portal (bhuvan.nrsc.gov.in) — India-specific GIS layers
Tertiary source	MNRE-NRSC SRRA dataset — district-level GHI (Global Horizontal Irradiance)
Key metrics	GHI (kWh/m2/day), DNI, peak sun hours, monthly averages, best/worst months
Granularity	District level for all 6 states (covers ~80 districts total)
Method	NASA: Python requests to REST API per lat-lon. ISRO: download GeoTIFF, parse with rasterio
Update cadence	Annually (climate averages are stable; no need for live feeds)

2.3 State Electricity Tariffs

Leh (UT)	JERC	Rs 3.50 – 5.00	jerc.nic.in
Maharashtra	MERC	Rs 7.00 – 12.00	mercindia.org.in
Gujarat	GERC	Rs 5.50 – 9.00	gercin.org
Rajasthan	RERC	Rs 6.00 – 9.50	rerc.rajasthan.gov.in
Karnataka	KERC	Rs 6.50 – 10.00	kerc.gov.in
Telangana	TSERC	Rs 6.00 – 9.00	tserc.gov.in

Note: Tariff ranges vary by slab and consumer category (LT commercial vs HT industrial). Download annual tariff orders as PDFs from each regulator and store in structured SQL.

2.4 Solar Panel Vendors

Field	Detail
Sources	MNRE approved vendor list (PDF), IndiaMART (scrape), Waaree / Tata / Adani Solar official sites
Scraper stack	Scrapy (async), BeautifulSoup for parsing, rotating proxies for IndiaMART
Key fields	Vendor name, state presence, panel watt-peak (Wp), price/Wp (Rs), warranty years, MNRE approved (Y/N), co
Update cadence	Quarterly (prices fluctuate ~5-10% per quarter)
Volume	~150-300 vendor records across 6 states

2.5 Maintenance Cost Benchmarks

Sourced from CEEW (ceew.in) reports, MNRE O&M; guidelines, and industry white papers. Key figures: Annual O&M; = Rs 0.30–0.50/Wp; panel degradation rate = 0.5% per year; cleaning cost = Rs 2–5/panel/month in dusty states (Rajasthan, Leh). Store as a static JSON config file — updates annually.

2.6 ROI Calculation Inputs

Variable	Formula / Source
System size (kWp)	Monthly consumption (kWh) / (Peak sun hrs/day x 30 x 0.8 performance ratio)
System cost (Rs)	System size x Avg cost/kWp (Rs 40,000–60,000) — state-specific
Annual generation (kWh)	System size x Peak sun hrs x 365 x 0.8
Annual savings (Rs)	Annual generation x State per-unit tariff
Net investment (Rs)	System cost minus applicable subsidy
Payback period (yrs)	Net investment / Annual savings
10-yr ROI (%)	(10 x Annual savings / Net investment) x 100

03

Data Storage Architecture

The project uses a hybrid storage model: unstructured policy text goes into a vector database for RAG retrieval, while structured numeric data (tariffs, vendor records, radiation values) lives in SQLite for fast filtered queries.

Gov scheme PDFs / text	Vector DB	ChromaDB	RAG — semantic search over policy docs
MNRE guidelines / circulars	Vector DB	ChromaDB	LLM needs to retrieve exact clause text
Electricity tariffs	SQL	SQLite	Structured lookup by state + consumer category
Solar radiation / GHI	SQL	SQLite	Numeric district-level lookup by state
Vendor records	SQL	SQLite	Filtered by state, MNRE approval, price
Maintenance benchmarks	JSON config	File	Small static reference, loaded at startup
ROI formula config	JSON config	File	Formulas + constants, versioned in git
Conversation memory	In-memory	LangChain Buffer	Session-scoped, not persisted

Vector Database — ChromaDB

All government scheme PDFs and MNRE circulars are chunked (500 tokens, 50 token overlap), embedded using **nomic-embed-text** via Ollama, and stored in a local ChromaDB collection. The scheme retrieval chain does a top-5 similarity search on the user query, feeds results into the LLM prompt, and the LLM synthesises a structured answer.

Parameter	Value
Embedding model	nomic-embed-text (Ollama — runs locally)
Chunk size	500 tokens, 50 token overlap
Collection name	solar_schemes
Retrieval strategy	Top-5 cosine similarity
Metadata stored	state, scheme_type (central/state), valid_until, source_url

SQL Database — SQLite Schema (Key Tables)

Table: vendors

Column	Type
`id`	`INTEGER PRIMARY KEY`
`name`	`TEXT`
`state`	`TEXT`
`watts_peak`	`INTEGER`
`price_per_wp_rs`	`REAL`
`warranty_years`	`INTEGER`

mnre_approved	BOOLEAN
website	TEXT

Table: tariffs

Column	Type
id	INTEGER PRIMARY KEY
state	TEXT
consumer_category	TEXT
rate_rs_per_unit	REAL
effective_date	DATE
source_url	TEXT

Table: radiation

Column	Type
id	INTEGER PRIMARY KEY
state	TEXT
district	TEXT
ghi_kwh_m2_day	REAL
peak_sun_hours	REAL
best_month	TEXT
worst_month	TEXT

04

Web UI Features & Design

The web interface is a React + TailwindCSS single-page app served by a FastAPI backend. The chat panel is the core interaction surface. Rich response cards, state selectors, and summary panels enhance usability for business users.

Core Chat Interface

Feature	Description
Multi-turn conversation	Full session memory — agent remembers state, business type, and previous answers within a session
Streaming responses	LLM tokens stream in real-time via Server-Sent Events (SSE) — no waiting for full response
Intent indicators	Small badge shows which chain handled the query (e.g. ROI Calculator, Scheme Retrieval)
Suggested follow-ups	3 clickable quick-reply chips after each response (e.g. 'Show vendor list', 'Recalculate with 200 kWh')
Chat history	Session log persists in localStorage; downloadable as PDF or JSON

State & Business Onboarding

Feature	Description
Smart onboarding flow	First message triggers a structured input panel: state dropdown, business type, monthly bill — pre-populates context
State selector	Visual card grid of the 6 covered states with solar potential badge (High / Medium)
Input validation	Validates electricity consumption and roof area before triggering ROI chain
Edit context anytime	Floating 'Edit Details' button lets user update state/load mid-conversation

Rich Response Cards

Feature	Description
ROI Summary Card	Visual card with: investment, subsidy deduction, net cost, annual savings, payback period, 10-yr ROI bar chart
Scheme Cards	Each scheme shown as a card: name, subsidy %, eligibility, validity, apply button
Vendor Table	Sortable table — filter by state, MNRE approved, price/Wp, warranty

Solar Heatmap	Embedded GHI heatmap for selected state (static SVG from pre-rendered ISRO data)
Maintenance Timeline	Year-by-year cost + degradation table for 10-year horizon

Utility Features

Feature	Description
Export to PDF	One-click export of the full session summary as a branded PDF report
WhatsApp share	Share ROI summary card via WhatsApp Web link (mobile-friendly)
Dark mode	Full dark/light toggle — important for field use on bright screens
Responsive layout	Mobile-first — works on tablet for on-site usage at business premises
Language toggle	Hindi / English toggle (basic i18n for wider business reach)

Admin / Developer Panel

Feature	Description
Data freshness indicator	Shows last-updated date for each data source (scheme data, tariffs, vendors)
Trigger re-scrape	Button to kick off vendor/scheme scraper jobs (protected by admin key)
RAG debug mode	Dev-only toggle shows retrieved document chunks alongside LLM response
Intent log	Logs which chain was invoked per query — useful for evaluating router accuracy

05

Tech Stack Summary

LLM (local)	Ollama + Llama 3 / Mistral 7B	Core language model, runs locally
Embeddings	nomic-embed-text via Ollama	Document embeddings for RAG
Agent framework	LangChain (Python)	Chains, memory, tool routing, RAG
Vector DB	ChromaDB (local)	Store & retrieve scheme/policy embeddings
Relational DB	SQLite	Vendors, tariffs, radiation data
Backend API	FastAPI (Python)	REST endpoints + SSE streaming
Frontend	React + TailwindCSS	Chat UI, rich response cards
Web scraping	Scrapy + BeautifulSoup	Vendor & scheme data collection
Radiation data	NASA POWER REST API + ISRO Bhuvan	GHI peak sun hours per district
PDF export	ReportLab (Python)	Session summary export
Hosting	Local / Railway.app	Dev on localhost, deploy on Railway

Project Folder Structure

```
solar-break/
■■■ backend/
■   ■■■ main.py                 # FastAPI app + SSE streaming
■   ■■■ agent/
■   ■   ■■■ router.py           # Intent router chain
■   ■   ■■■ chains/
■   ■   ■   ■■■ scheme_chain.py  # RAG over gov docs
■   ■   ■   ■■■ roi_chain.py     # ROI calculator tool
■   ■   ■   ■■■ vendor_chain.py  # SQL vendor lookup
■   ■   ■   ■■■ viability_chain.py  # Radiation data lookup
■   ■   ■■■ memory.py           # ConversationBufferMemory
■   ■■■ data/
■   ■   ■■■ ingest/
■   ■   ■   ■■■ scheme_scraper.py
■   ■   ■   ■■■ vendor_scraper.py
■   ■   ■   ■■■ radiation_fetcher.py
■   ■   ■■■ vector_store/       # ChromaDB persisted here
■   ■   ■■■ solar_break.db      # SQLite database
■   ■■■ config/
■       ■■■ maintenance.json
■       ■■■ roi_config.json
■■■ frontend/
    ■■■ src/
    ■   ■■■ components/
    ■   ■   ■■■ ChatPanel.jsx
    ■   ■   ■■■ ROICard.jsx
    ■   ■   ■■■ SchemeCard.jsx
    ■   ■   ■■■ VendorTable.jsx
    ■   ■   ■■■ StateSelector.jsx
    ■   ■■■ App.jsx
    ■■■ tailwind.config.js
```

Solar Break — University Project | LangChain + Ollama | October 2026